SENATI

Escuela de Tecnologías de la Información

GUÍA DE LABORATORIO

Guía 16

Entropía y el Algoritmo ID3

Construye algoritmo de árbol de decisiones

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Módulo formativo	Artificial Intelligence with Machine Learning in Java (Oracle Academy)
Semestre	IV
Semana	16
Tarea de aprendizaje	Construye algoritmo de árbol de decisiones
Herramientas	Java 21 (JDK) · Visual Studio Code · Extension Pack for Java
Instructor	Rembrandt Ubalde Enríquez
Estudiante	
Fecha	

Contenido

Objetivos de la guía 3
Materiales 3
Parte 1. Preparación del entorno en Visual Studio Code 4
1.1 Verificar o instalar el JDK 4
1.2 Instalar la extensión de Java en VS Code 4
1.3 Opción A: abrir el proyecto entregado 4
1.4 Opción B: crear el proyecto desde cero 4
1.5 Ejecutar los programas 5
Parte 2. Conceptos clave 6
2.1 Algoritmos de árboles de decisión 6
2.2 Entropía de la información 6
2.3 Ganancia de información 6
2.4 El algoritmo ID3 7
2.5 Ejemplo resuelto de ID3: ¿Se juega tenis? 7
2.6 Limitaciones de ID3 9
Parte 3. Prácticas en Visual Studio Code 10
Práctica 01. Entropía de la información 10
Práctica 02. Ganancia de información 11
Práctica 03. Crear un árbol ID3 14
Práctica 04. Proyecto: evaluador de créditos con ID3 18
Parte 4. Errores frecuentes 23
Parte 5. Ejercicios propuestos 23
Preguntas de reflexión 23
Lista de cotejo 24

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Objetivos de la guía

Al terminar esta guía serás capaz de:

- **Usar** algoritmos de árboles de decisión para clasificar datos.
- **Aplicar** la entropía y la ganancia de información para elegir la mejor pregunta.
- **Resolver** un problema con el algoritmo ID3, a mano y programado en Java.
- **Crear** un árbol ID3 con tus propios datos y evaluarlo.

Operación	Dónde se trabaja en esta guía
Usa algoritmos de árboles de decisión	Parte 2 (sección 2.1), Prácticas 03 y 04
Aplica entropía y ganancia de información	Parte 2 (secciones 2.2 y 2.3), Prácticas 01 y 02
Resuelve un problema con el algoritmo ID3	Parte 2 (secciones 2.4 y 2.5), Práctica 03
Crear un árbol ID3	Prácticas 03 y 04

Materiales

- Computadora con Windows 10/11, **JDK 17 o superior** y Visual Studio Code con **Extension Pack for Java**.
- Proyecto **Semana16_ID3_Java.zip** entregado por el instructor.
- Calculadora científica para el ejemplo resuelto a mano.

Parte 1. Preparación del entorno en Visual Studio Code

Esta guía trae el proyecto de VS Code **Semana16_ID3_Java** con todas las clases listas. Puedes abrirlo directamente (opción A) o crearlo tú desde cero y escribir cada clase (opción B, recomendada para practicar).

1.1 Verificar o instalar el JDK

Abre una terminal (PowerShell) y escribe:

Terminal de VS Code

```
java -version
javac -version
```

▶ Resultado esperado (versión 17 o superior)

```
java version "21.0.x" ...
javac 21.0.x
```

Si aparece **«java no se reconoce como un comando»**, instala el **JDK 21** (Oracle JDK desde oracle.com/java o Eclipse Temurin desde adoptium.net). Durante la instalación marca las opciones **Add to PATH** y **Set JAVA_HOME**, y luego cierra y vuelve a abrir VS Code.

> ⚠ **Atención:** Los programas de esta guía usan `record`, `switch` con flechas y otras novedades de Java moderno. Necesitas **Java 17 o superior**; con Java 8 o 11 aparecerán errores de compilación.

1.2 Instalar la extensión de Java en VS Code

1. En VS Code abre la vista de **Extensiones** (Ctrl + Shift + X).
2. Busca **Extension Pack for Java** (de Microsoft) e instálala. Incluye el soporte del lenguaje, el depurador, el ejecutor de pruebas y el administrador de proyectos.
3. Reinicia VS Code si te lo solicita.

1.3 Opción A: abrir el proyecto entregado

1. Descomprime **Semana16_ID3_Java.zip** en tu carpeta **Documentos**.
2. En VS Code entra a **Archivo > Abrir carpeta…** y selecciona la carpeta **Semana16_ID3_Java** (la que contiene `src`, no la carpeta de arriba).
3. Elige **Sí, confío en los autores** si VS Code lo pregunta.
4. Espera a que en la barra inferior aparezca **Java: Ready** (la primera vez puede tardar un minuto).

1.4 Opción B: crear el proyecto desde cero

1. Pulsa **Ctrl + Shift + P**, escribe **Java: Create Java Project** y elige **No build tools**.
2. Selecciona **Documentos** como ubicación y escribe el nombre **Semana16_ID3_Java**.
3. VS Code crea las carpetas `src`, `lib`, `bin` y el archivo `src/App.java` (puedes borrarlo).

4. Crea una carpeta **datos** junto a `src`. En la carpeta `src` crea cada clase con clic derecho > **Nuevo archivo**, usando el nombre exacto que indica la guía.

La estructura final del proyecto debe quedar así:

Estructura del proyecto

```
Semana16_ID3_Java/
├── .vscode/settings.json
├── datos/
│   ├── jugar_tenis.csv        (14 días: ejemplo clásico de Quinlan)
│   └── creditos.csv           (24 solicitudes de crédito: proyecto)
└── src/
    ├── Datos.java             (carga archivos CSV)
    ├── NodoID3.java           (nodo con varias ramas)
    ├── ID3.java               (el algoritmo)
    ├── P01_Entropia.java
    ├── P02_GananciaInformacion.java
    ├── P03_ID3Tenis.java
    └── P04_ProyectoCreditos.java
```

⚠ **Atención:** En Java el **nombre del archivo** debe ser idéntico al de la clase pública, respetando mayúsculas: la clase `public class P01_DatosInformacion` va en `P01_DatosInformacion.java`.

1.5 Ejecutar los programas

1. Abre el archivo (por ejemplo `P01_Entropia.java`).
2. Haz clic en **Run** que aparece encima de `public static void main`, o en el botón ▶ de arriba a la derecha.
3. El resultado aparece en la **TERMINAL**. Si el programa pide datos con `Scanner`, escríbelos allí y presiona Enter.
4. Compara lo que ves con el recuadro verde **Resultado esperado**.

También puedes compilar y ejecutar desde la terminal, ubicado en la carpeta del proyecto:

Terminal de VS Code

```
javac -encoding UTF-8 -d bin src\*.java
java -cp bin P01_Entropia
```

💡 **Consejo:** Si las tildes o la ñ se ven como símbolos raros (por ejemplo `INFORMACI?N`), escribe una vez `chcp 65001` en la terminal y vuelve a ejecutar.

💡 **Consejo:** Para depurar: haz clic a la izquierda del número de línea para poner un **punto de interrupción** (punto rojo) y elige **Debug** encima del `main`. Con **F10** avanzas línea por línea y con **F11** entras a un método; el panel **Variables** muestra los valores en cada momento.

Parte 2. Conceptos clave

2.1 Algoritmos de árboles de decisión

Un **árbol de decisión** es un modelo de aprendizaje **supervisado** de **clasificación** que se parece al juego Sí/No de las semanas 14-15, con una gran diferencia: **las preguntas no las escribe una persona**, el algoritmo las elige a partir de los datos.

Elemento	Significado
Nodo de decisión	Pregunta sobre un **atributo** (¿Cómo está el cielo?).
Rama	Un **valor** posible del atributo (Soleado, Nublado, Lluvioso). Puede haber más de dos.
Hoja	La **clase** predicha (Jugar = Sí).
Camino raíz → hoja	Una **regla**: SI Cielo = Soleado Y Humedad = Alta ENTONCES No.

Ventajas: son fáciles de entender y explicar (cada predicción se justifica con una regla), funcionan con datos categóricos y son la base de modelos más potentes como **Random Forest**. Los algoritmos más conocidos son **ID3** (Quinlan, 1986), su sucesor **C4.5** y **CART**.

2.2 Entropía de la información

La **entropía** (Claude Shannon, 1948) mide el **desorden** o la **incertidumbre** de un conjunto de datos. Si todos los ejemplos tienen la misma clase, no hay incertidumbre (entropía 0). Si las clases están mezcladas por igual, la incertidumbre es máxima.

Fórmula de la entropía

$$H(S) = -\sum p_i \cdot \log_2(p_i)$$

p_i = proporción de ejemplos de la clase i en el conjunto S
$\log_2(x) = \ln(x) / \ln(2)$ (en Java: `Math.log(x) / Math.log(2)`)

Conjunto	Entropía	Interpretación
14 Sí, 0 No	0.000	Puro: no hay nada que decidir.
13 Sí, 1 No	0.371	Casi puro.
9 Sí, 5 No	0.940	Bastante mezclado.
7 Sí, 7 No	1.000	Máximo desorden con dos clases.

2.3 Ganancia de información

La **ganancia de información** mide **cuánto se reduce la entropía** al separar los datos según un atributo. La mejor pregunta es la que deja los grupos más «puros».

Fórmula de la ganancia

$$\text{Ganancia}(S, A) = H(S) - \Sigma \left(|S_v| / |S| \right) \cdot H(S_v)$$

S_v = subconjunto de S donde el atributo A tiene el valor v
$|S_v| / |S|$ = peso del subconjunto (qué parte del total representa)

2.4 El algoritmo ID3

ID3 (**I**terative **D**ichotomiser **3**) construye el árbol de arriba hacia abajo de forma **recursiva**, tal como los métodos de árboles de la semana anterior:

Pseudocódigo de ID3

```
ID3(filas, atributos):
  1. Si todas las filas tienen la misma clase -> HOJA con esa clase    (caso base)
  2. Si no quedan atributos                   -> HOJA con la clase mayoritaria
  3. Calcular la ganancia de cada atributo y elegir el MEJOR (mayor ganancia)
  4. Crear un NODO que pregunta por el mejor atributo
  5. Para cada valor v del mejor atributo:
       subárbol = ID3(filas con valor v, atributos sin el mejor)    (recursión)
       agregar una rama v -> subárbol
  6. Devolver el NODO
```

2.5 Ejemplo resuelto de ID3: ¿Se juega tenis?

Este es el ejemplo clásico de Quinlan. Durante 14 días se registró el clima y si se jugó tenis. **Objetivo:** construir un árbol que prediga si se jugará según el clima.

Archivo: datos/jugar_tenis.csv

```
Cielo,Temperatura,Humedad,Viento,Jugar
Soleado,Caluroso,Alta,Débil,No
Soleado,Caluroso,Alta,Fuerte,No
Nublado,Caluroso,Alta,Débil,Sí
Lluvioso,Templado,Alta,Débil,Sí
Lluvioso,Frío,Normal,Débil,Sí
Lluvioso,Frío,Normal,Fuerte,No
Nublado,Frío,Normal,Fuerte,Sí
Soleado,Templado,Alta,Débil,No
Soleado,Frío,Normal,Débil,Sí
Lluvioso,Templado,Normal,Débil,Sí
Soleado,Templado,Normal,Fuerte,Sí
Nublado,Templado,Alta,Fuerte,Sí
Nublado,Caluroso,Normal,Débil,Sí
Lluvioso,Templado,Alta,Fuerte,No
```

Paso 1. Entropía del conjunto completo

Cálculo

9 Sí y 5 No de 14 días

$$H(S) = -(9/14)\cdot\log_2(9/14) - (5/14)\cdot\log_2(5/14)$$
$$= -(0.643)(-0.637) - (0.357)(-1.485)$$
$$= 0.410 + 0.530 = 0.940$$

Paso 2. Ganancia de cada atributo

Atributo	Grupos (Sí / No) y entropía	Entropía ponderada	Ganancia
Cielo	Soleado 2/3 (0.971) · Nublado 4/0 (0) · Lluvioso 3/2 (0.971)	5/14·0.971 + 4/14·0 + 5/14·0.971 = 0.694	**0.247**
Temperatura	Caluroso 2/2 (1) · Templado 4/2 (0.918) · Frío 3/1 (0.811)	0.911	0.029
Humedad	Alta 3/4 (0.985) · Normal 6/1 (0.592)	0.788	0.152
Viento	Débil 6/2 (0.811) · Fuerte 3/3 (1)	0.892	0.048

Cielo tiene la mayor ganancia, así que es la **raíz**. La rama **Nublado** es pura (4 Sí): se convierte directamente en una hoja **Sí**.

Paso 3. Rama Soleado (5 días: 2 Sí, 3 No, H = 0.971)

Atributo	Grupos (Sí / No)	Ganancia
Temperatura	Caluroso 0/2 · Templado 1/1 · Frío 1/0	0.971 − 0.400 = 0.571
Humedad	Alta 0/3 · Normal 2/0 (ambos puros)	0.971 − 0 = **0.971**
Viento	Débil 1/2 · Fuerte 1/1	0.971 − 0.951 = 0.020

Paso 4. Rama Lluvioso (5 días: 3 Sí, 2 No, H = 0.971)

Atributo	Grupos (Sí / No)	Ganancia
Temperatura	Templado 2/1 · Frío 1/1	0.020
Humedad	Alta 1/1 · Normal 2/1	0.020
Viento	Débil 3/0 · Fuerte 0/2 (ambos puros)	**0.971**

Paso 5. Árbol final

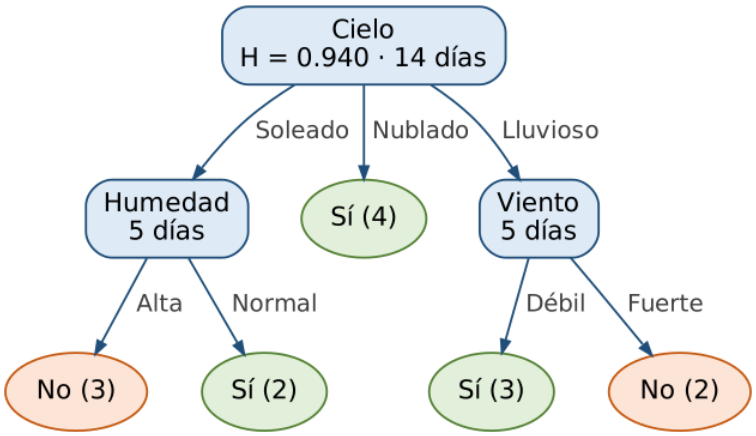


Figura 1. Árbol ID3 del ejemplo «¿Se juega tenis?» (entre paréntesis, número de días).

Con solo 3 preguntas el árbol clasifica correctamente los 14 días. Observa que **Temperatura no aparece**: ID3 descubrió que no aporta información útil.

2.6 Limitaciones de ID3

- Solo trabaja con atributos **categóricos**; los numéricos (edad, precio) deben agruparse en rangos. **C4.5** resuelve esto.
- Puede **sobreajustarse**: crea ramas para casos muy específicos que no se repiten en datos nuevos. Se corrige con la **poda** del árbol.
- Prefiere atributos con muchos valores distintos (por ejemplo, un DNI tendría ganancia máxima y no sirve para predecir).

Parte 3. Prácticas en Visual Studio Code

Práctica 01. Entropía de la información

Archivo: src/P01_Entropia.java

```
// P01_Entropia.java - Entropía de la información: mide el DESORDEN de un conjunto
import java.util.Locale;

public class P01_Entropia {

  // H(S) = - Σ p_i · log2(p_i)
  static double entropia(int... conteos) {
    int total = 0;
    for (int c : conteos) total += c;
    double h = 0;
    for (int c : conteos) {
      if (c == 0) continue;                    // 0 · log2(0) se toma como 0
      double p = (double) c / total;
      h -= p * log2(p);
    }
    return h;
  }

  static double log2(double x) {
    return Math.log(x) / Math.log(2);  // Java no trae log2: se cambia de base
  }

  public static void main(String[] args) {
    Locale.setDefault(Locale.US);
    int[][] casos = {{14, 0}, {13, 1}, {10, 4}, {9, 5}, {7, 7}, {5, 5, 5}};
    System.out.println("Conjunto (conteo por clase)   Entropía   Interpretación");
    for (int[] c : casos) {
      double h = entropia(c);
      String txt = (h == 0) ? "PURO: todos iguales"
           : (h >= 0.99 && c.length == 2) ? "MÁXIMO desorden (50/50)"
           : (c.length == 3) ? "3 clases iguales: máximo = log2(3)"
           : (h < 0.5) ? "casi puro" : "mezclado";
      System.out.printf("%-28s %8.3f   %s%n", java.util.Arrays.toString(c), h, txt);
    }

    System.out.println("\nCálculo paso a paso para [9 Sí, 5 No]:");
    double p1 = 9 / 14.0, p2 = 5 / 14.0;
    System.out.printf("  p(Sí) = 9/14 = %.4f   ->  -p·log2(p) = %.4f%n", p1,
        -p1 * log2(p1));
    System.out.printf("  p(No) = 5/14 = %.4f   ->  -p·log2(p) = %.4f%n", p2,
        -p2 * log2(p2));
    System.out.printf("  H(S)  = %.4f + %.4f = %.4f bits%n",
        -p1 * log2(p1), -p2 * log2(p2), entropia(9, 5));
  }
}
```

▶ **Resultado esperado en la terminal**

```
Conjunto (conteo por clase)   Entropía   Interpretación
[14, 0]                         0.000   PURO: todos iguales
[13, 1]                         0.371   casi puro
[10, 4]                         0.863   mezclado
[9, 5]                          0.940   mezclado
[7, 7]                          1.000   MÁXIMO desorden (50/50)
[5, 5, 5]                       1.585   3 clases iguales: máximo = log2(3)

Cálculo paso a paso para [9 Sí, 5 No]:
  p(Sí) = 9/14 = 0.6429   ->  -p·log2(p) = 0.4098
  p(No) = 5/14 = 0.3571   ->  -p·log2(p) = 0.5305
  H(S)  = 0.4098 + 0.5305 = 0.9403 bits
```

Análisis: compara los resultados con la tabla de la sección 2.2 y con el Paso 1 del ejemplo resuelto. Con 3 clases iguales la entropía máxima es $\log_2(3) = 1.585$, mayor que 1.

Práctica 02. Ganancia de información

Primero crea la clase `Datos`, que carga un CSV y se reutiliza en el resto de prácticas:

Archivo: src/Datos.java

```
// Datos.java - Carga un archivo CSV: la última columna es la CLASE (etiqueta)
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Datos {
  public final List<String> atributos;  // nombres de las columnas (sin la clase)
  public final String nombreClase;          // nombre de la última columna
  public final List<String[]> filas;  // cada fila: valores de atributos + clase

  public Datos(List<String> atributos, String nombreClase, List<String[]> filas) {
    this.atributos = atributos;
    this.nombreClase = nombreClase;
    this.filas = filas;
  }

  public static Datos cargarCSV(String ruta) throws IOException {
    List<String> lineas = Files.readAllLines(Path.of(ruta));
    String[] cabecera = lineas.get(0).trim().split(",");
    List<String[]> filas = new ArrayList<>();
    for (int i = 1; i < lineas.size(); i++) {
      if (lineas.get(i).isBlank()) continue;
      String[] f = lineas.get(i).trim().split(",");
      for (int j = 0; j < f.length; j++) f[j] = f[j].trim();
      filas.add(f);
    }
    List<String> atributos = new ArrayList<>(Arrays.asList(cabecera).subList(0,
        cabecera.length - 1));
    return new Datos(atributos, cabecera[cabecera.length - 1], filas);
  }

  public int columna(String atributo) {
    return atributos.indexOf(atributo);
  }

  public static String clase(String[] fila) {
    return fila[fila.length - 1];
  }
}
```

Archivo: src/P02_GananciaInformacion.java

```
// P02_GananciaInformacion.java - ¿Qué atributo separa MEJOR los datos?
import java.io.IOException;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.TreeMap;

public class P02_GananciaInformacion {

  // Entropía de un grupo de filas según su clase (última columna)
  static double entropia(List<String[]> filas) {
    Map<String, Integer> conteo = new TreeMap<>();
    for (String[] f : filas) conteo.merge(Datos.clase(f), 1, Integer::sum);
    double h = 0;
    for (int c : conteo.values()) {
      double p = (double) c / filas.size();
      h -= p * Math.log(p) / Math.log(2);
    }
    return h;
  }

  public static void main(String[] args) throws IOException {
    Locale.setDefault(Locale.US);
    Datos d = Datos.cargarCSV("datos/jugar_tenis.csv");
    double hTotal = entropia(d.filas);
    System.out.printf("Entropía del conjunto completo H(S) = %.3f%n", hTotal);

    String mejor = null;
    double mejorGanancia = -1;
    for (String atributo : d.atributos) {
      int col = d.columna(atributo);
      // Separar las filas según el valor del atributo
      Map<String, List<String[]>> grupos = new TreeMap<>();
      for (String[] f : d.filas) {
        grupos.computeIfAbsent(f[col], k -> new java.util.ArrayList<>()).add(f);
      }
      System.out.println("\nAtributo: " + atributo);
      double hPonderada = 0;
      for (var g : grupos.entrySet()) {
        long si = g.getValue().stream()
            .filter(f -> Datos.clase(f).equals("Sí")).count();
        long no = g.getValue().size() - si;
        double h = entropia(g.getValue());
        double peso = (double) g.getValue().size() / d.filas.size();
        hPonderada += peso * h;
        System.out.printf("   %-9s %2d filas [%d Sí, %d No]  H = %.3f  peso = %d/14%n",
            g.getKey(), g.getValue().size(), si, no, h, g.getValue().size());
      }
      double ganancia = hTotal - hPonderada;
      System.out.printf("   Ganancia(%s) = %.3f - %.3f = %.3f%n", atributo, hTotal,
          hPonderada, ganancia);
      if (ganancia > mejorGanancia) { mejorGanancia = ganancia; mejor = atributo; }
    }
    System.out.printf("%nMEJOR ATRIBUTO para la raíz: %s (ganancia = %.3f)%n", mejor,
        mejorGanancia);
  }
}
```

▶ **Resultado esperado en la terminal**

```
Entropía del conjunto completo H(S) = 0.940

Atributo: Cielo
   Lluvioso    5 filas [3 Sí, 2 No]  H = 0.971  peso = 5/14
   Nublado     4 filas [4 Sí, 0 No]  H = 0.000  peso = 4/14
   Soleado     5 filas [2 Sí, 3 No]  H = 0.971  peso = 5/14
   Ganancia(Cielo) = 0.940 - 0.694 = 0.247

Atributo: Temperatura
   Caluroso    4 filas [2 Sí, 2 No]  H = 1.000  peso = 4/14
   Frío        4 filas [3 Sí, 1 No]  H = 0.811  peso = 4/14
   Templado    6 filas [4 Sí, 2 No]  H = 0.918  peso = 6/14
   Ganancia(Temperatura) = 0.940 - 0.911 = 0.029

Atributo: Humedad
   Alta        7 filas [3 Sí, 4 No]  H = 0.985  peso = 7/14
   Normal      7 filas [6 Sí, 1 No]  H = 0.592  peso = 7/14
   Ganancia(Humedad) = 0.940 - 0.788 = 0.152

Atributo: Viento
   Débil       8 filas [6 Sí, 2 No]  H = 0.811  peso = 8/14
   Fuerte      6 filas [3 Sí, 3 No]  H = 1.000  peso = 6/14
   Ganancia(Viento) = 0.940 - 0.892 = 0.048

MEJOR ATRIBUTO para la raíz: Cielo (ganancia = 0.247)
```

Análisis: el programa reproduce exactamente el **Paso 2** del ejemplo resuelto. El atributo con más ganancia es **Cielo (0.247)** y por eso será la raíz del árbol.

Práctica 03. Crear un árbol ID3

Ahora programarás el algoritmo completo. Crea primero `NodoID3`: a diferencia del `Nodo` binario de las semanas 14-15, guarda sus hijos en un `Map` porque un atributo puede tener **más de dos valores**.

Archivo: src/NodoID3.java

```
// NodoID3.java - Nodo de un árbol de decisión (puede tener VARIAS ramas)
import java.util.LinkedHashMap;
import java.util.Map;

public class NodoID3 {
  String atributo;  // pregunta del nodo (null si es hoja)
  String clase;                                    // respuesta, solo en las hojas
  String mayoria;  // clase más frecuente en este nodo
  Map<String, NodoID3> hijos = new LinkedHashMap<>(); // valor del atributo -> subárbol

  boolean esHoja() {
    return atributo == null;
  }
}
```

La clase `ID3` sigue el pseudocódigo de la sección 2.4. Identifica en el código los **dos casos base** y la **llamada recursiva** dentro del método `construir`:

Archivo: src/ID3.java

```
// ID3.java - Algoritmo ID3 (Iterative Dichotomiser 3) de Ross Quinlan
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.TreeMap;

public class ID3 {
  private final Datos datos;
  private NodoID3 raiz;
  public boolean mostrarPasos = false;

  public ID3(Datos datos) {
    this.datos = datos;
  }

  public NodoID3 entrenar() {
    raiz = construir(datos.filas, new ArrayList<>(datos.atributos), 0);
    return raiz;
  }

  // ===== Construcción RECURSIVA del árbol =====
  private NodoID3 construir(List<String[]> filas, List<String> atributos, int nivel) {
    NodoID3 nodo = new NodoID3();
    nodo.mayoria = claseMayoritaria(filas);

    // CASO BASE 1: todas las filas tienen la misma clase -> hoja pura
    if (entropia(filas) == 0) {
      nodo.clase = Datos.clase(filas.get(0));
      return nodo;
    }
    // CASO BASE 2: ya no quedan atributos -> hoja con la clase mayoritaria
    if (atributos.isEmpty()) {
      nodo.clase = nodo.mayoria;
      return nodo;
    }
    // PASO RECURSIVO: elegir el atributo con MAYOR ganancia de información
    String mejor = null;
    double mejorG = -1;
    for (String a : atributos) {
      double g = ganancia(filas, a);
      if (mostrarPasos) System.out.printf(Locale.US, "%sGanancia(%s) = %.3f%n",
          "   ".repeat(nivel), a, g);
      if (g > mejorG) { mejorG = g; mejor = a; }
    }
    if (mostrarPasos) System.out.println("   ".repeat(nivel) + "=> se elige: " + mejor);
    nodo.atributo = mejor;

    List<String> restantes = new ArrayList<>(atributos);
    restantes.remove(mejor);
    for (var g : separar(filas, mejor).entrySet()) {
      if (mostrarPasos) System.out.println("   ".repeat(nivel) + "Rama " + mejor
          + " = " + g.getKey()
                        + " (" + g.getValue().size() + " filas)");
      nodo.hijos.put(g.getKey(), construir(g.getValue(), restantes, nivel + 1));
    }
    return nodo;
  }

  // ===== Fórmulas =====
  public static double entropia(List<String[]> filas) {
    Map<String, Integer> conteo = new TreeMap<>();
    for (String[] f : filas) conteo.merge(Datos.clase(f), 1, Integer::sum);
    double h = 0;
    for (int c : conteo.values()) {
      double p = (double) c / filas.size();
      h -= p * Math.log(p) / Math.log(2);
```

```
    }
    return h;
  }

  public double ganancia(List<String[]> filas, String atributo) {
    double hPonderada = 0;
    for (List<String[]> grupo : separar(filas, atributo).values()) {
      hPonderada += (double) grupo.size() / filas.size() * entropia(grupo);
    }
    return entropia(filas) - hPonderada;
  }

  private Map<String, List<String[]>> separar(List<String[]> filas, String atributo) {
    int col = datos.columna(atributo);
    Map<String, List<String[]>> grupos = new TreeMap<>();
    for (String[] f : filas) grupos.computeIfAbsent(f[col],
        k -> new ArrayList<>()).add(f);
    return grupos;
  }

  private static String claseMayoritaria(List<String[]> filas) {
    Map<String, Integer> conteo = new TreeMap<>();
    for (String[] f : filas) conteo.merge(Datos.clase(f), 1, Integer::sum);
    return Collections.max(conteo.entrySet(), Map.Entry.comparingByValue()).getKey();
  }

  // ===== Clasificar un ejemplo nuevo: bajar por el árbol (recursivo) =====
  public String clasificar(String[] ejemplo) {
    return clasificar(raiz, ejemplo);
  }
  private String clasificar(NodoID3 n, String[] ejemplo) {
    if (n.esHoja()) return n.clase;
    String valor = ejemplo[datos.columna(n.atributo)];
    NodoID3 hijo = n.hijos.get(valor);
    if (hijo == null) return n.mayoria;  // valor nunca visto: usa la mayoría
    return clasificar(hijo, ejemplo);
  }

  // ===== Mostrar el árbol (recorrido PRE-ORDER) =====
  public void imprimir() {
    System.out.println("[" + raiz.atributo + "]");
    imprimir(raiz, "");
  }
  private void imprimir(NodoID3 n, String prefijo) {
    int i = 0;
    for (var e : n.hijos.entrySet()) {
      boolean ultimo = (++i == n.hijos.size());
      NodoID3 h = e.getValue();
      String texto = h.esHoja() ? "=> " + datos.nombreClase + ": "
          + h.clase.toUpperCase()
                : "[" + h.atributo + "]";
      System.out.println(prefijo + (ultimo ? "└── " : "├── ") + e.getKey() + " "
          + texto);
      if (!h.esHoja()) imprimir(h, prefijo + (ultimo ? "    " : "│   "));
    }
  }

  // ===== Convertir el árbol en reglas SI ... ENTONCES =====
  public void imprimirReglas() {
    reglas(raiz, "");
  }
  private void reglas(NodoID3 n, String condicion) {
    if (n.esHoja()) {
      System.out.println("SI " + condicion + " ENTONCES " + datos.nombreClase + " = "
          + n.clase);
      return;
    }
    for (var e : n.hijos.entrySet()) {
      String c = n.atributo + " = " + e.getKey();
      reglas(e.getValue(), condicion.isEmpty() ? c : condicion + " Y " + c);
```

```
    }
  }
}
```

Método	Qué hace	Técnica
`construir`	Crea el árbol eligiendo el atributo de mayor ganancia.	Recursividad
`entropia` / `ganancia`	Aplican las fórmulas de las secciones 2.2 y 2.3.	Map para contar
`clasificar`	Baja por el árbol siguiendo los valores del ejemplo hasta una hoja.	Recursividad
`imprimir`	Dibuja el árbol con ramas.	Recorrido **pre-order**
`imprimirReglas`	Convierte cada camino raíz → hoja en una regla SI … ENTONCES.	Recorrido **pre-order**

Finalmente, el programa que entrena el árbol con los datos del tenis:

Archivo: src/P03_ID3Tenis.java

```
// P03_ID3Tenis.java - Crear un árbol ID3 y usarlo para predecir
import java.io.IOException;

public class P03_ID3Tenis {
  public static void main(String[] args) throws IOException {
    Datos datos = Datos.cargarCSV("datos/jugar_tenis.csv");
    System.out.println("Datos: " + datos.filas.size() + " días | Atributos: "
        + datos.atributos);

    ID3 id3 = new ID3(datos);
    id3.mostrarPasos = true;  // muestra cómo decide en cada nivel
    System.out.println("\n--- Construcción del árbol ---");
    id3.entrenar();

    System.out.println("\n--- Árbol de decisión obtenido ---");
    id3.imprimir();

    System.out.println("\n--- Reglas SI ... ENTONCES ---");
    id3.imprimirReglas();

    System.out.println("\n--- Predicción de días nuevos ---");
    String[][] nuevos = {
      {"Soleado", "Frío", "Alta", "Fuerte"},
      {"Lluvioso", "Caluroso", "Normal", "Débil"},
      {"Nublado", "Frío", "Alta", "Fuerte"},
    };
    for (String[] dia : nuevos) {
      System.out.println(String.join(", ", dia) + "  ->  ¿Jugar? "
          + id3.clasificar(dia));
    }
  }
}
```

▶ **Resultado esperado en la terminal**

```
Datos: 14 días | Atributos: [Cielo, Temperatura, Humedad, Viento]

--- Construcción del árbol ---
Ganancia(Cielo) = 0.247
Ganancia(Temperatura) = 0.029
Ganancia(Humedad) = 0.152
Ganancia(Viento) = 0.048
=> se elige: Cielo
Rama Cielo = Lluvioso (5 filas)
   Ganancia(Temperatura) = 0.020
   Ganancia(Humedad) = 0.020
   Ganancia(Viento) = 0.971
   => se elige: Viento
   Rama Viento = Débil (3 filas)
   Rama Viento = Fuerte (2 filas)
Rama Cielo = Nublado (4 filas)
Rama Cielo = Soleado (5 filas)
   Ganancia(Temperatura) = 0.571
   Ganancia(Humedad) = 0.971
   Ganancia(Viento) = 0.020
   => se elige: Humedad
   Rama Humedad = Alta (3 filas)
   Rama Humedad = Normal (2 filas)

--- Árbol de decisión obtenido ---
[Cielo]
├── Lluvioso [Viento]
│    ├── Débil => Jugar: SÍ
│    └── Fuerte => Jugar: NO
├── Nublado => Jugar: SÍ
└── Soleado [Humedad]
     ├── Alta => Jugar: NO
     └── Normal => Jugar: SÍ

--- Reglas SI ... ENTONCES ---
SI Cielo = Lluvioso Y Viento = Débil ENTONCES Jugar = Sí
SI Cielo = Lluvioso Y Viento = Fuerte ENTONCES Jugar = No
SI Cielo = Nublado ENTONCES Jugar = Sí
SI Cielo = Soleado Y Humedad = Alta ENTONCES Jugar = No
SI Cielo = Soleado Y Humedad = Normal ENTONCES Jugar = Sí

--- Predicción de días nuevos ---
Soleado, Frío, Alta, Fuerte  ->  ¿Jugar? No
Lluvioso, Caluroso, Normal, Débil  ->  ¿Jugar? Sí
Nublado, Frío, Alta, Fuerte  ->  ¿Jugar? Sí
```

Análisis: gracias a `mostrarPasos = true` ves cómo ID3 calcula las ganancias en cada nivel, exactamente como en los Pasos 2, 3 y 4 del ejemplo resuelto. El árbol coincide con la Figura 1. El tercer día nuevo (Nublado) se clasifica como **Sí** sin mirar los demás atributos.

Práctica 04. Proyecto: evaluador de créditos con ID3

Una cooperativa quiere automatizar la aprobación de créditos usando su historial de 24 solicitudes. Usarás la **misma clase ID3** con otro dataset: así se comprueba que el algoritmo es general y **aprende las reglas de cualquier problema**.

Paso 1. Crear el archivo de datos

Archivo: datos/creditos.csv

```
Ingreso,Deudas,Empleo,Historial,Aprobar
Alto,No,Estable,Bueno,Sí
Alto,Sí,Estable,Bueno,Sí
Alto,No,Temporal,Bueno,Sí
Alto,Sí,Temporal,Malo,No
Alto,No,Estable,Malo,Sí
Medio,No,Estable,Bueno,Sí
Medio,Sí,Estable,Bueno,Sí
Medio,Sí,Temporal,Bueno,No
Medio,No,Temporal,Bueno,Sí
Medio,Sí,Estable,Malo,No
Medio,No,Temporal,Malo,No
Bajo,No,Estable,Bueno,Sí
Bajo,Sí,Estable,Bueno,No
Bajo,No,Temporal,Bueno,Sí
Bajo,Sí,Temporal,Bueno,No
Bajo,No,Estable,Malo,No
Bajo,Sí,Temporal,Malo,No
Alto,Sí,Estable,Malo,No
Medio,No,Estable,Malo,Sí
Bajo,Sí,Estable,Malo,No
Alto,Sí,Temporal,Bueno,Sí
Medio,Sí,Temporal,Malo,No
Alto,No,Temporal,Malo,Sí
Bajo,No,Temporal,Malo,No
```

Paso 2. Crear el programa

Archivo: src/P04_ProyectoCreditos.java

```
// P04_ProyectoCreditos.java - PROYECTO: sistema que aprueba créditos con ID3
import java.io.IOException;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Random;
import java.util.Scanner;

public class P04_ProyectoCreditos {
  public static void main(String[] args) throws IOException {
    Locale.setDefault(Locale.US);
    Datos todos = Datos.cargarCSV("datos/creditos.csv");
    System.out.println("=== EVALUADOR DE CRÉDITOS (ID3) ===");
    System.out.println("Solicitudes históricas: " + todos.filas.size());

    // 1. Dividir: 75 % entrenamiento, 25 % prueba (semilla fija)
    List<String[]> filas = new ArrayList<>(todos.filas);
    Collections.shuffle(filas, new Random(7));
    int corte = (int) (filas.size() * 0.75);
    Datos train = new Datos(todos.atributos, todos.nombreClase, filas.subList(0,
        corte));
    List<String[]> test = filas.subList(corte, filas.size());
    System.out.println("Entrenamiento: " + train.filas.size() + " | Prueba: "
        + test.size());

    // 2. Entrenar el árbol
    ID3 id3 = new ID3(train);
    id3.entrenar();
    System.out.println("\nÁrbol aprendido:");
    id3.imprimir();

    // 3. Evaluar con datos que el árbol no vio
    int aciertos = 0;
    System.out.println("\nEvaluación:");
    for (String[] f : test) {
      String pred = id3.clasificar(f);
      String real = Datos.clase(f);
      if (pred.equals(real)) aciertos++;
      System.out.printf("  %-28s real=%-2s pred=%-2s %s%n",
          String.join(",", java.util.Arrays.copyOf(f, f.length - 1)),
          real, pred, pred.equals(real) ? "acierto" : "ERROR");
    }
    System.out.printf("Exactitud: %d/%d = %.1f %%%n", aciertos, test.size(),
        100.0 * aciertos / test.size());

    // 4. Usar el sistema con un cliente nuevo
    Scanner sc = new Scanner(System.in);
    String[] cliente = new String[todos.atributos.size()];
    String[][] opciones = {
        {"Alto", "Medio", "Bajo"}, {"Sí", "No"},
        {"Estable", "Temporal"}, {"Bueno", "Malo"}};
    System.out.println("\nNuevo cliente:");
    for (int i = 0; i < cliente.length; i++) {
      System.out.print("  " + todos.atributos.get(i) + " " + String.join("/",
          opciones[i]) + ": ");
      cliente[i] = sc.nextLine().trim();
    }
    System.out.println("DECISIÓN: " + (id3.clasificar(cliente).equals("Sí")
        ? "CRÉDITO APROBADO" : "CRÉDITO DENEGADO"));
  }
}
```

Paso 3. Ejecutar y evaluar dos clientes

▶ **Resultado esperado (cliente: Medio, Sí, Estable, Bueno)**

```
=== EVALUADOR DE CRÉDITOS (ID3) ===
Solicitudes históricas: 24
Entrenamiento: 18 | Prueba: 6

Árbol aprendido:
[Deudas]
├── No [Historial]
│   ├── Bueno => Aprobar: SÍ
│   └── Malo [Ingreso]
│       ├── Alto => Aprobar: SÍ
│       ├── Bajo => Aprobar: NO
│       └── Medio [Empleo]
│           ├── Estable => Aprobar: SÍ
│           └── Temporal => Aprobar: NO
└── Sí [Ingreso]
    ├── Alto [Historial]
    │   ├── Bueno => Aprobar: SÍ
    │   └── Malo => Aprobar: NO
    ├── Bajo => Aprobar: NO
    └── Medio [Empleo]
        ├── Estable => Aprobar: SÍ
        └── Temporal => Aprobar: NO

Evaluación:
  Medio,Sí,Estable,Malo        real=No pred=Sí ERROR
  Alto,No,Estable,Bueno        real=Sí pred=Sí acierto
  Bajo,Sí,Temporal,Malo        real=No pred=No acierto
  Medio,Sí,Temporal,Malo       real=No pred=No acierto
  Alto,Sí,Temporal,Bueno       real=Sí pred=Sí acierto
  Alto,No,Estable,Malo         real=Sí pred=Sí acierto
Exactitud: 5/6 = 83.3 %

Nuevo cliente:
  Ingreso Alto/Medio/Bajo: Medio
  Deudas Sí/No: Sí
  Empleo Estable/Temporal: Estable
  Historial Bueno/Malo: Bueno
DECISIÓN: CRÉDITO APROBADO
```

▶ **Resultado esperado (cliente: Bajo, Sí, Temporal, Bueno)**

```
=== EVALUADOR DE CRÉDITOS (ID3) ===
Solicitudes históricas: 24
Entrenamiento: 18 | Prueba: 6

Árbol aprendido:
[Deudas]
├── No [Historial]
│   ├── Bueno => Aprobar: SÍ
│   └── Malo [Ingreso]
│       ├── Alto => Aprobar: SÍ
│       ├── Bajo => Aprobar: NO
│       └── Medio [Empleo]
│           ├── Estable => Aprobar: SÍ
│           └── Temporal => Aprobar: NO
└── Sí [Ingreso]
    ├── Alto [Historial]
    │   ├── Bueno => Aprobar: SÍ
    │   └── Malo => Aprobar: NO
    ├── Bajo => Aprobar: NO
    └── Medio [Empleo]
        ├── Estable => Aprobar: SÍ
        └── Temporal => Aprobar: NO

Evaluación:
  Medio,Sí,Estable,Malo        real=No pred=Sí ERROR
  Alto,No,Estable,Bueno        real=Sí pred=Sí acierto
  Bajo,Sí,Temporal,Malo        real=No pred=No acierto
  Medio,Sí,Temporal,Malo       real=No pred=No acierto
  Alto,Sí,Temporal,Bueno       real=Sí pred=Sí acierto
  Alto,No,Estable,Malo         real=Sí pred=Sí acierto
Exactitud: 5/6 = 83.3 %

Nuevo cliente:
  Ingreso Alto/Medio/Bajo: Bajo
  Deudas Sí/No: Sí
  Empleo Estable/Temporal: Temporal
  Historial Bueno/Malo: Bueno
DECISIÓN: CRÉDITO DENEGADO
```

Análisis: con los datos de la cooperativa, ID3 eligió **Deudas** como la pregunta más importante. En los datos de prueba acertó **5 de 6** (83.3 %). El error ocurrió con un cliente de ingreso medio, con deudas y mal historial: el árbol no pregunta por el historial en esa rama porque, entre los 18 ejemplos de entrenamiento, no había un caso igual. Con **más datos** el árbol mejoraría.

> ⚠ **Atención:** Escribe los valores **exactamente** como en el CSV, con mayúscula inicial y tilde (`Sí`, `Bueno`). Si escribes `si`, el árbol no encuentra la rama y responde con la clase mayoritaria de ese nodo.

Parte 4. Errores frecuentes

Mensaje o síntoma	Causa	Solución
La entropía sale `NaN`	Se calculó `0 * log2(0)`.	Omite las clases con conteo 0 (`if (c == 0) continue;`).
`ArrayIndexOutOfBoundsException` al leer el CSV	Una línea tiene menos columnas o hay una línea mal escrita.	Revisa que todas las filas tengan 5 valores separados por comas.
El árbol predice siempre lo mismo	Los valores del ejemplo no coinciden con los del CSV (`si` vs `Sí`).	Usa exactamente los mismos textos; revisa espacios y tildes.
Las tildes del CSV se leen mal	El archivo no se guardó en UTF-8.	En VS Code, abajo a la derecha, elige **UTF-8 > Guardar con codificación**.
`NoSuchFileException: datos\jugar_tenis.csv`	Se abrió otra carpeta en VS Code.	Abre la carpeta **Semana16_ID3_Java**.
`cannot find symbol: class ID3`	Falta crear `ID3.java`, `Datos.java` o `NodoID3.java`.	Crea las tres clases en `src` antes de ejecutar las prácticas.

Parte 5. Ejercicios propuestos

1. Calcula **a mano** la entropía de los conjuntos [6 Sí, 2 No], [4 Sí, 4 No] y [3, 3, 2] (tres clases). Verifica tus resultados agregándolos a la Práctica 01.
2. Adapta la Práctica 02 para calcular la ganancia de los 4 atributos de `creditos.csv` con las 24 filas. ¿Qué atributo sería la raíz? ¿Coincide con el árbol de la Práctica 04? Explica por qué podría ser diferente.
3. Agrega al archivo `jugar_tenis.csv` el día 15: `Nublado, Templado, Alta, Débil, No`. Vuelve a ejecutar la Práctica 03. ¿Cómo cambia el árbol?
4. Crea tu propio dataset `aprobacion.csv` con al menos 14 filas y los atributos Asistencia (Alta/Media/Baja), Tareas (Completas/Incompletas), Estudia (Sí/No) y la clase Aprueba (Sí/No). Construye el árbol con ID3 y muestra sus reglas.
5. Modifica la Práctica 04 para mostrar la **matriz de confusión**: verdaderos positivos, verdaderos negativos, falsos positivos (créditos aprobados que debían denegarse) y falsos negativos. ¿Qué error es más costoso para la cooperativa?

Preguntas de reflexión

1. ¿Por qué un atributo con ganancia 0 no sirve como pregunta en el árbol?
2. ¿Qué relación tiene el algoritmo ID3 con la recursividad y los recorridos que aprendiste en las semanas 14-15?
3. ¿Por qué un árbol que acierta el 100 % de los datos de entrenamiento podría fallar con datos nuevos?
4. ¿Por qué los árboles de decisión se consideran modelos «explicables»? ¿En qué casos es importante poder explicar una decisión de la IA?

Lista de cotejo

- ☐ Explico qué es un árbol de decisión y sus elementos.
- ☐ Calculo la entropía de un conjunto a mano y en Java (Práctica 01).
- ☐ Calculo la ganancia de información de cada atributo (Práctica 02).
- ☐ Resolví a mano el ejemplo del tenis siguiendo los pasos de ID3.
- ☐ Programé el algoritmo ID3 e identifico sus casos base y su llamada recursiva (Práctica 03).
- ☐ Obtuve las reglas SI … ENTONCES de un árbol.
- ☐ Entrené y evalué un árbol ID3 con otro dataset (Práctica 04).
- ☐ Resolví al menos tres ejercicios propuestos.